Glossary

Level 3 words, from A to Z, in plain language

Dyslexia-friendly edition (Lexend, wider spacing).

Tip: How to use this glossary

Words are in alphabetical order. The week number tells you where to read more in your workbook. Missing a word? Write it on the last lines with your own definition.

algorithm A precise list of steps that solves a problem, like a recipe. Binary search and selection sort are famous ones. (week 4)

attribute A value stored inside an object, such as `self.name` or `self.energy`. What an object **has**. (week 2)

average (mean) Add up all the numbers and divide by how many there are. (week 8)

base case The part of a recursive function that stops it from calling itself again. Without one you get a `RecursionError`. (week 7)

bias When data or a program treats some people or things unfairly, often because the training data left some of them out. (week 9)

binary search Searching a **sorted** list by checking the middle and throwing

away the half that cannot hold the answer. Very fast. (week 4)

breadth-first search (BFS) Exploring a maze in rings, one step further each time, using a queue. It finds a shortest path. (week 7)

bubble sort Sorting by swapping neighbors that are in the wrong order, again and again. (week 5)

class A blueprint for making objects. It lists their attributes and methods. (week 2)

class diagram A drawing of the classes in a program, their attributes and methods, and how they are connected. (week 10)

classifier A program that puts things into groups (labels), such as “dragon” or “slime”. (week 9)

clean (data) Fix messy data before using it: remove empty rows, strip spaces, turn text into numbers. (week 8)

code review Reading someone’s code to find bugs and suggest improvements, kindly. (week 10)

commit A save point in git, with a message saying what changed. (week 1)

composition Building an object out of other objects: a Game **has a** Maze and a Player. (week 3)

CRC card A paper card for designing a class: its name (Class), its jobs (Responsibilities) and who it works with (Collaborators). (week 10)

CSV “Comma-separated values”: a table saved as plain text, one row per line, commas between columns. (week 8)

data structure A way of organizing data in a program: a list, dictionary, stack, queue or grid. (week 5)

debugger A tool (the bug button in Thonny) that runs your program one step at a time so you can watch the variables. (week 6)

deepfake A fake picture, video or voice made by AI to look real. (week 9)

deque “Double-ended queue” from `collections`. Fast to add at the back and take from the front with `popleft()`. (week 5)

diff The lines that changed between two versions. `git diff` shows them. (week 1)

edge case An unusual input that can break a program, such as an empty list. Good tests try edge cases. (week 11)

feature (AI) A number that describes an example, such as size or number of spikes. Classifiers compare features. (week 9)

flood fill Filling every connected square of the same kind, like the paint bucket tool. Usually written with recursion. (week 7)

fractal A pattern made of smaller copies of itself, like a tree whose branches look like little trees. (week 7)

git A program that keeps save points (commits) of a folder so you can see and undo changes. (week 1)

grid (2D list) A list of lists, like a chessboard: `grid[row][col]`. (week 7)

inheritance When a class gets all the code of a parent class and adds or changes some: a Knight **is a** Character. (week 3)

`__init__` The method that runs when a new object is made. It sets the starting attributes. (week 2)

license The rules that say how you may use someone's code, pictures or sounds. (week 10)

linear search Checking items one by one from the start until you find what you want. (week 4)

Markov text generator A program that writes new text by choosing each next word from the words that followed it in its training text. (week 9)

method A function that belongs to a class. What an object **does**. (week 2)

milestone A checkpoint in a project plan, with a date and a list of what should work. (week 10)

nearest neighbor An AI method: label a mystery item with the label of the most similar training example. (week 9)

object (instance) One thing made from a class: `mo = Pet("Mo")` makes one

pet. (week 2)

override Writing a method in a child class with the same name as in the parent class, to change what it does. (week 3)

polymorphism Different classes answering the same method call in their own way. (week 3)

queue First in, first out, like a lunch line. (week 5)

README A file that explains what a project is, how to run it and how it is built. (week 11)

recursion When a function calls itself on a smaller version of the same problem. (week 7)

RecursionError Python's error when a function calls itself too many times, usually because the base case is missing. (week 7)

repository (repo) A folder that git is tracking, with all its save points. (week 1)

restore `git restore file.py` throws away changes you have not committed yet. (week 1)

`self` Inside a method, the object the method was called on. (week 2)

selection sort Sorting by finding the smallest item and swapping it to the front, then repeating with the rest. (week 5)

stack Last in, first out, like a pile of plates or an undo button. (week 5)

`super()` Calls the parent class's version of a method, often `super().__init__(…)`. (week 3)

Terminal The Mac app where you type commands such as `git status`. (week 1)

training data The labeled examples an AI learns from. (week 9)

More words from your workbook

accessibility Designing so that people with different abilities can use it too. (week 11)

accuracy Correct guesses divided by the number of test examples. (week 9)

anonymize Remove or replace anything that could show who gave an answer. (week 8)

breakpoint A marked line where the debugger pauses. (week 6)

code of conduct The rules a community agrees to, so everyone stays safe and respected. (week 12)

commit id The short code name of a commit, like `a1b2c3d`. (week 1)

commit message A short label that says what changed, like “Add jump to hero”. (week 1)

consent Agreeing, freely, to take part – with the choice to say no. (week 8)

data cleaning Fixing messy data (spaces, capitals, missing or empty values) before using it. (week 8)

design decision A choice about how to build something, with a reason. (week 12)

feedback Comments that help someone improve: kind, specific, useful. (week 12)

header The first line of a CSV file, with the column names. (week 8)

key function A function that tells `sorted` what to sort by. (week 5)

off-by-one error A bug where a number is 1 too big or 1 too small, like using < instead of <=. (week 4)

parent class / child class The general class and the more special class built from it. Also called superclass and subclass. (week 3)

playtest Watching someone use your project to find problems. (week 11)

recursive case The part that does a small step and calls the function on a smaller problem. (week 7)

sample The group of people who answered. A small sample can give misleading results. (week 8)

showcase An event where makers show and explain their work. (week 12)

step into Go inside the function or method that the line calls. (week 6)

step over Run the highlighted line in one go, then pause. (week 6)

test data Examples kept secret during training, used to check the guesses. (week 9)

trace table A paper table where you write each variable's value, step by step. (week 6)

traceback The error report Python prints. Read it from the bottom. (week 6)

triage Sorting problems by how much they matter, to fix the important ones first. (week 11)

usability How easy something is to use for someone new. (week 11)

version 2 The next, improved release of a project. (week 12)

worst case The unluckiest situation, when an algorithm needs the most steps. (week 4)

My own words